

Introducing Retrieval Augmented Generation

Retrieval Augmented Generation (RAG) is a technique that helps you work with large documents that are too big to fit into a single prompt. Instead of cramming everything into one massive prompt, RAG breaks documents into chunks and only includes the most relevant pieces when answering questions.

The Problem with Large Documents

Imagine you have an 800-page financial document and want to ask Claude specific questions about it, like "What risk factors does this company have?" You need to get the relevant information from the document to Claude somehow, but there are limits to how much text you can include in a prompt.

Claude with large documents

- We might want to ask Claude questions about specific sections of a large document
 - "What risk factors does this company have?"
- It's a private document - Claude won't have any idea what is contained in it
- How can we get Claude to understand the contents?

ANTHROPIC

Financial Doc, 800+ pages

Corporation Annual Financial Report - Fiscal Year 2024
1. Executive Summary Corporation ("the Company") is pleased to present its Annual Financial Report for the fiscal year ended December 31, 2024. The Company has demonstrated strong financial performance across all key metrics, with revenue growth of 12.3% year-over-year, reaching \$1.24 billion. Operating income increased by 15.7% to \$342 million, while net income rose by 18.2% to \$267 million. The Board of Directors has approved a quarterly dividend of \$0.45 per share, representing a 10% increase from the previous quarter. This dividend will be payable on March 15, 2025, to shareholders of record as of February 28, 2025. The Company continues to invest in research and development, with R&D expenses increasing by 22.5% to \$156 million. These investments are focused on expanding our product portfolio and enhancing our technological capabilities to maintain our competitive edge in the market.
2. Management Discussion & Analysis Overview of Business Operations Corporation operates in three primary business segments: Consumer Products, Enterprise Solutions, and Digital Services. The Consumer Products segment, which accounts for 45% of total revenue, experienced growth of 9.8% year-over-year. The Enterprise Solutions segment, representing 35% of total revenue, grew by 14.2%. The Digital Services segment, which accounts for 20% of total revenue, demonstrated the strongest growth at 18.7%. Market Conditions and Competitive Landscape The global market for our products and services remained competitive throughout the fiscal year. We observed increased competition in the Consumer Products segment, particularly in emerging markets. However, our strategic focus on premium offerings and brand differentiation allowed us to maintain our market share and improve profit margins. Risk Factors The Company continues to monitor several risk factors that could impact future performance, including: • Fluctuations in raw material costs and supply chain disruptions • Changes in regulatory requirements across our operating regions • Technological disruptions and cybersecurity threats • Economic uncertainty in key markets • Foreign exchange rate volatility Future Outlook Looking ahead to fiscal year 2025, we anticipate continued growth across all business segments. We expect revenue growth of 10-12%, driven by new product launches and expansion into new geographic markets. We plan to maintain our focus on operational efficiency.

Option 1: Include Everything in the Prompt

The first approach is straightforward - extract all text from the document and stuff it into your prompt along with the user's question. Your prompt might look like this:

Answer the user's question about the financial document.

```
<user_question>
{user_question}
</user_question>
```

```
<financial_document>
{financial_document}
```

```
</financial_document>
```

Possibly not the best solution

- There's a hard limit on how much text we can feed into Claude - the document might be too long
- Claude gets slightly less effective with very long prompts
- Larger prompts **cost more** to process
- Larger prompts **take longer** to process

ANTHROPIC

Option 1: Include the contents of the document in a prompt

Corporate Financial Report - Q4 2023

1. Executive Summary

Overview: The Company's Q4 2023 performance is detailed in this report. The report covers key financial metrics, operational performance, and strategic initiatives. The report is structured as follows:

- **Section 1:** Executive Summary
- **Section 2:** Management Discussion & Analysis
- **Section 3:** Financial Statements
- **Section 4:** Risk Factors
- **Section 5:** Future Outlook

2. Management Discussion & Analysis

Overview of Business Operations

The Company's business operations are divided into three main segments: Consumer Products, Enterprise Solutions, and Digital Services. The Consumer Products segment, which accounts for 45% of total revenue, experienced growth of 12% year-over-year. The Enterprise Solutions segment, which accounts for 35% of total revenue, saw a decline of 5% year-over-year. The Digital Services segment, which accounts for 20% of total revenue, grew by 18% year-over-year.

Market Conditions and Competitive Landscape

The global market for consumer products and enterprise solutions remained competitive throughout the quarter. The Consumer Products segment faced increased competition from new entrants, while the Enterprise Solutions segment saw a shift in demand towards digital services.

Risk Factors

The Company's business operations are subject to various risks, including:

- **Market Risk:** Changes in market conditions, including interest rates and foreign exchange rates, could impact the Company's financial performance.
- **Operational Risk:** Disruptions in the Company's supply chain or operational processes could impact its ability to deliver products and services.
- **Financial Risk:** Changes in the Company's financial position, including its debt and equity structure, could impact its ability to fund operations.
- **Reputational Risk:** Negative publicity or legal proceedings could impact the Company's reputation and financial performance.

Future Outlook

Looking ahead, the Company expects continued growth across all segments. We expect revenue growth of 10-12% in 2024, driven by strong performance in the Consumer Products and Digital Services segments. We also expect to continue investing in research and development to drive innovation and growth.

Answer the user's question about the financial document.

```
<user_question>
{user_question}
</user_question>

<financial_document>
{financial_document}
</financial_document>
```

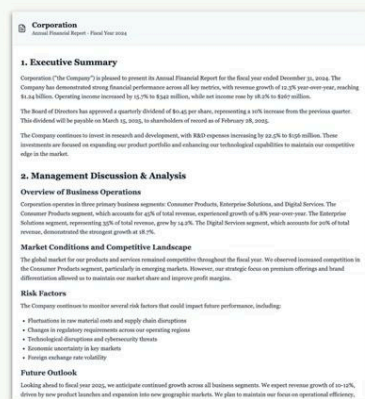
This approach has serious limitations:

- There's a hard limit on prompt length - your document might be too long
- Claude becomes less effective with very long prompts
- Larger prompts cost more to process
- Larger prompts take longer to process

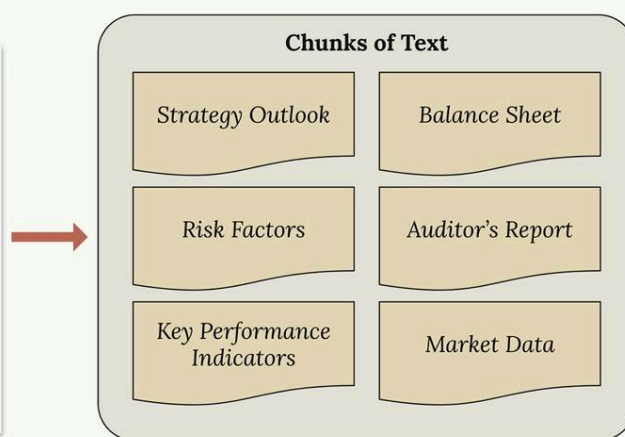
Option 2: Break Documents into Chunks

RAG takes a smarter approach. First, you break the document into smaller chunks during a preprocessing step. Then, when a user asks a question, you find the chunks most relevant to their question and only include those in your prompt.

Option 2: Break the document up into many chunks. Put chunks relevant to the user's question in the prompt.

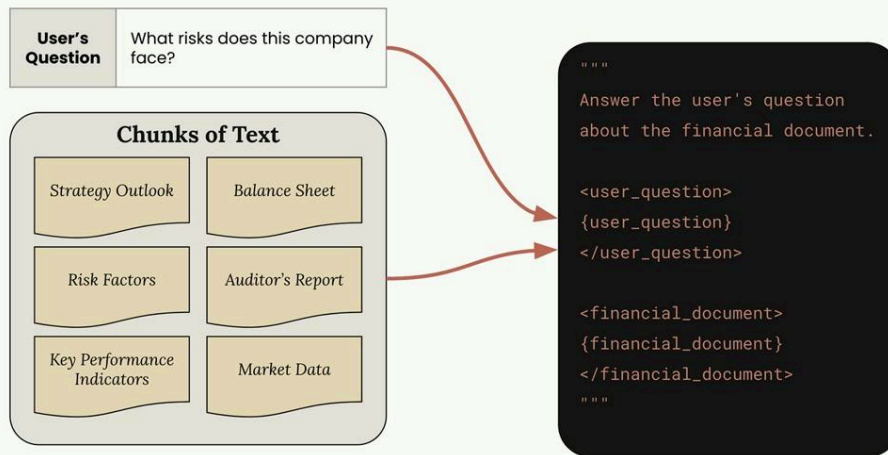


ANTHROPIC



Here's how it works: if someone asks "What risks does this company face?" you'd search through your chunks, find the "Risk Factors" section, and include just that relevant chunk in your prompt.

Option 2: Break the document up into many chunks. Put chunks relevant to the user's question in the prompt.



Benefits of RAG

- Claude can focus on only the most relevant content
- Scales up to very large documents
- Works with multiple documents
- Smaller prompts cost less and run faster

Challenges with RAG

- Requires a preprocessing step to chunk documents
- Need a search mechanism to find "relevant" chunks
- Included chunks might not contain all the context Claude needs
- Many ways to chunk text - which approach is best?

For example, you could split documents into equal-sized portions, or you could create chunks based on document structure like headers and sections. Each approach has trade-offs you'll need to evaluate for your specific use case.

When to Use RAG

RAG involves many technical decisions and requires more work than simply including everything in a prompt. You'll need to analyze whether the benefits outweigh the complexity for your particular application. It's especially valuable when working with very large documents, multiple documents, or when you need to optimize for cost and performance.

The key insight is that RAG trades simplicity for scalability and efficiency. While it requires more upfront work to implement properly, it enables you to work with document collections that would be impossible to handle with simple prompt stuffing.

Text chunking strategies

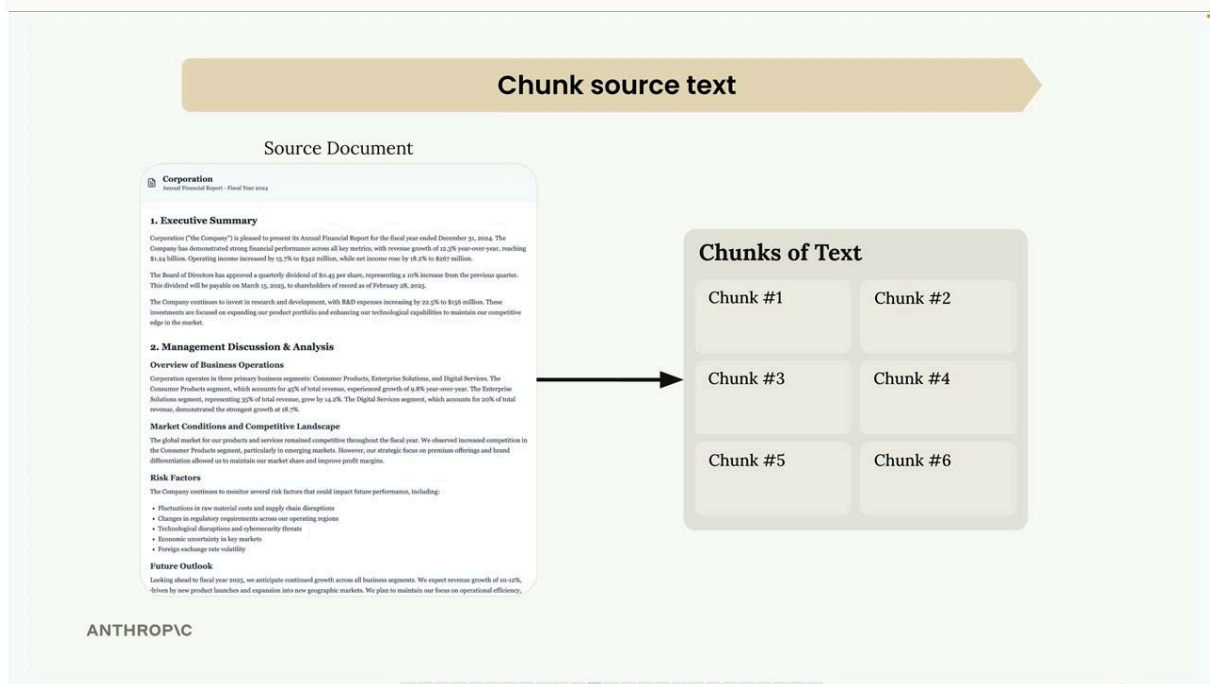
Downloads

001_chunking.ipynb

(opens in new tab)

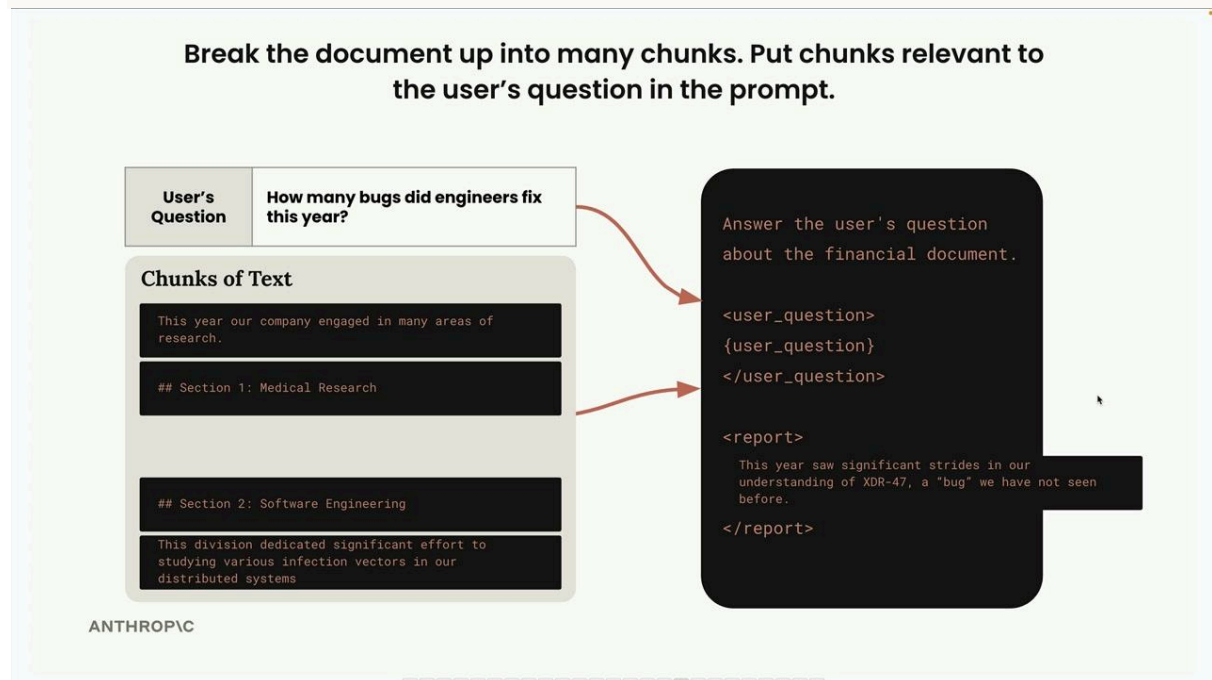
report.md

Text chunking is one of the most critical steps in building a RAG (Retrieval Augmented Generation) pipeline. How you break up your documents directly impacts the quality of your entire system. A poor chunking strategy can lead to irrelevant context being inserted into your prompts, causing your AI to give completely wrong answers.



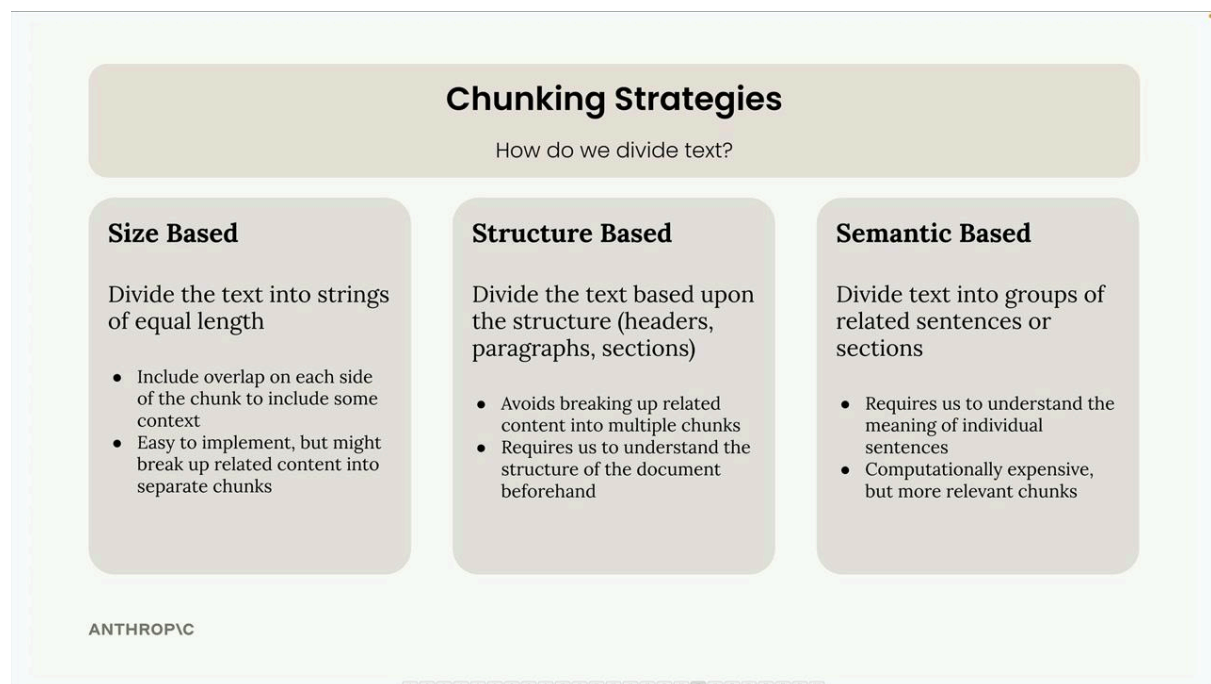
Consider this example: you have a document with sections on medical research and software engineering. If you chunk poorly, a user asking "How many bugs did engineers fix this year?" might get information about

medical research instead of software engineering, simply because the medical section happened to contain the word "bug" in a different context.

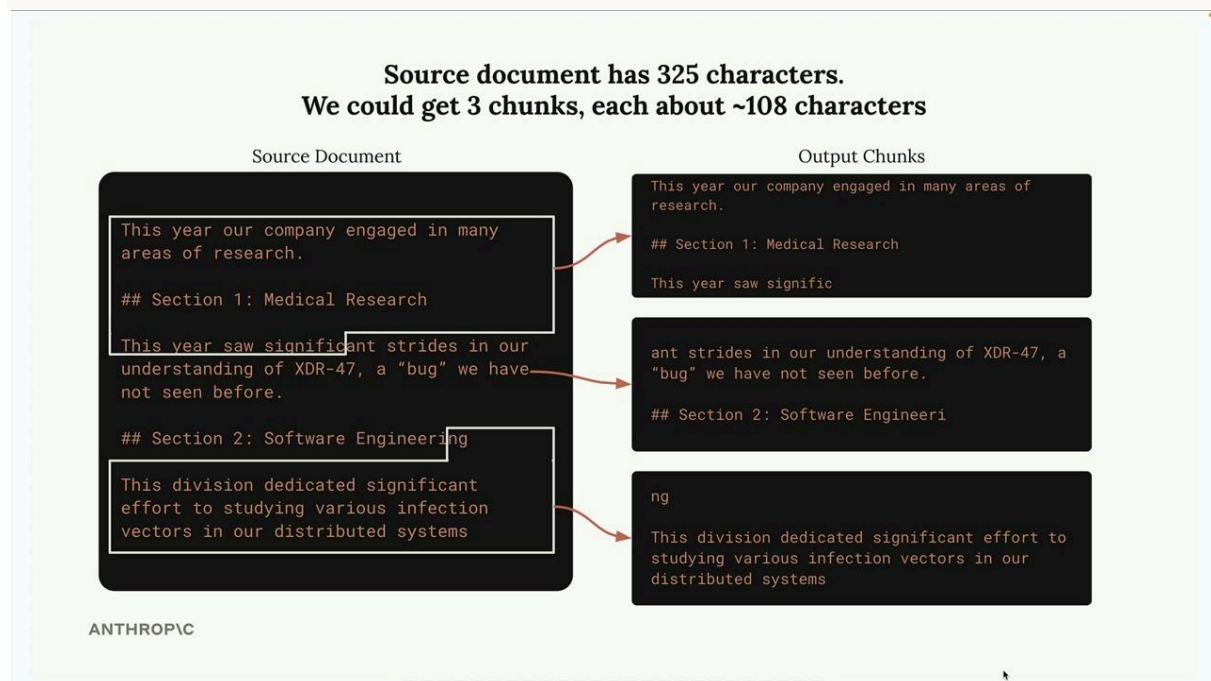


This is why choosing the right chunking strategy matters so much. Let's explore three main approaches.

Size-Based Chunking



Size-based chunking is the simplest approach - you divide your text into strings of equal length. If you have a 325-character document, you might split it into three chunks of roughly 108 characters each.



This method is easy to implement and works with any type of document, but it has clear downsides:

- Words get cut off mid-sentence
- Chunks lose important context from surrounding text
- Section headers might be separated from their content

Sized Based Downsides

- Each chunk has some cutoff text
- Each chunk lacks context

```
This year our company engaged in many areas of
research.

## Section 1: Medical Research

This year saw signific

ant strides in our understanding of XDR-47, a
"bug" we have not seen before.

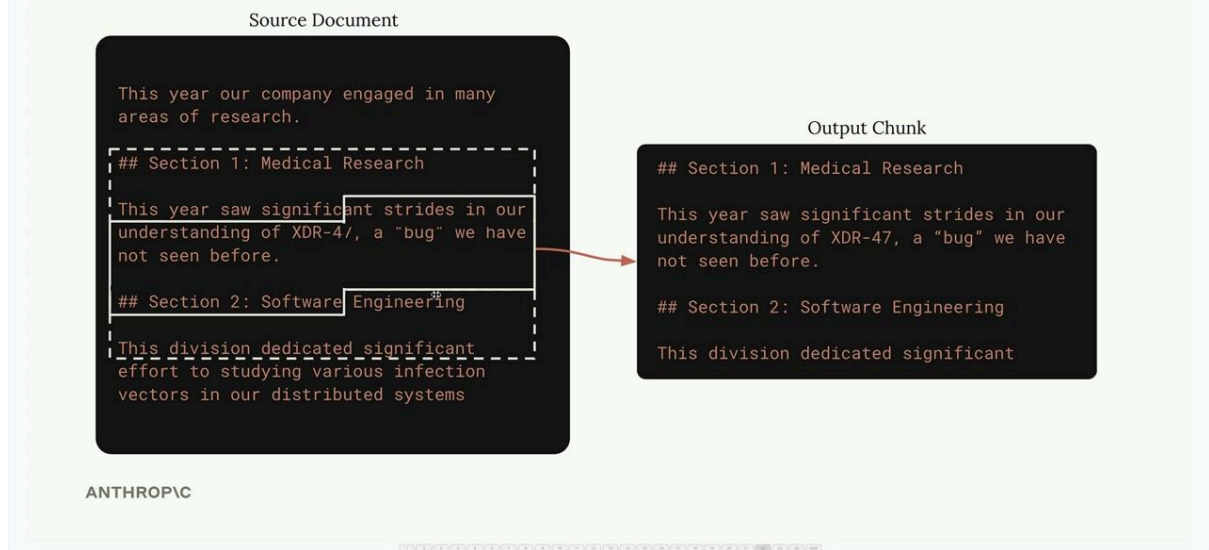
## Section 2: Software Engineeri

ng
This division dedicated significant effort to
studying various infection vectors in our
distributed systems
```

ANTHROPIC

To address these issues, you can add overlap between chunks. This means each chunk includes some characters from the neighboring chunks, providing better context and ensuring complete words and sentences.

Workaround: Each chunk gets some overlap with neighboring chunks



Here's a basic implementation:

```
def chunk_by_char(text, chunk_size=150, chunk_overlap=20):
    chunks = []
    start_idx = 0

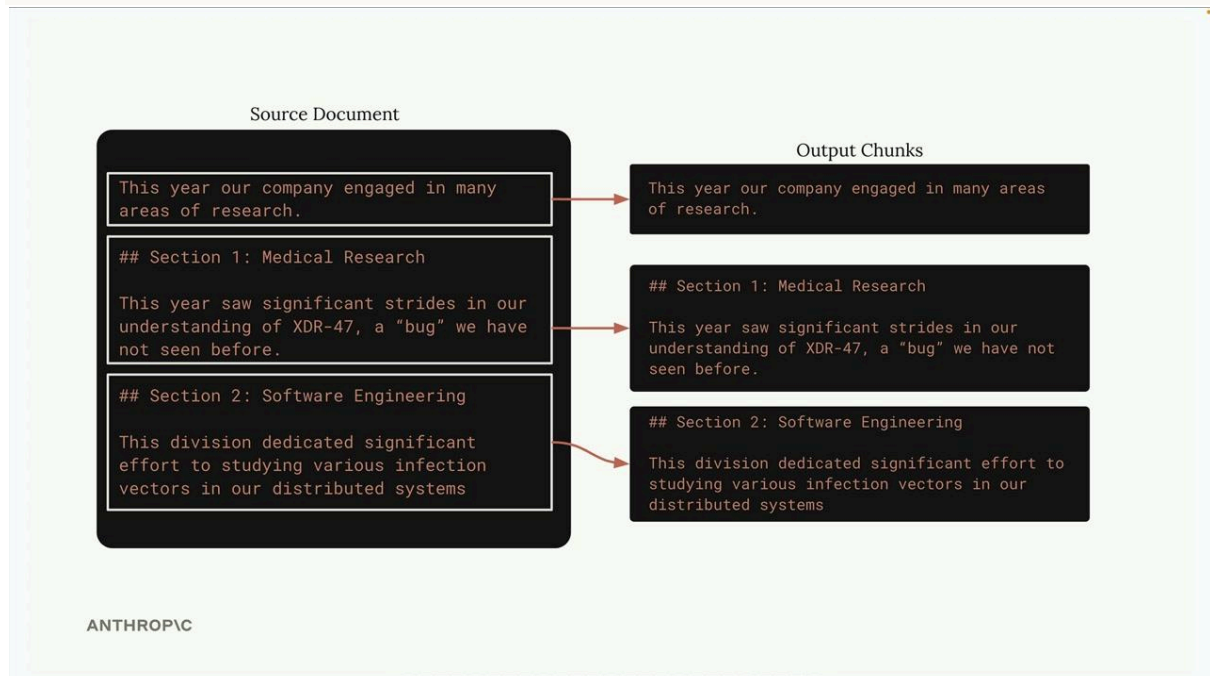
    while start_idx < len(text):
        end_idx = min(start_idx + chunk_size, len(text))
        chunk_text = text[start_idx:end_idx]
        chunks.append(chunk_text)

        start_idx = (
            end_idx - chunk_overlap if end_idx < len(text)
            else len(text)
        )

    return chunks
```

Structure-Based Chunking

Structure-based chunking divides text based on the document's natural structure - headers, paragraphs, and sections. This works great when you have well-formatted documents like Markdown files.



For a Markdown document, you can split on header markers:

```
def chunk_by_section(document_text):  
    pattern = r"\n## "  
  
    return re.split(pattern, document_text)
```

This approach gives you the cleanest, most meaningful chunks because each one represents a complete section. However, it only works when you have guarantees about your document structure. Many real-world documents are plain text or PDFs without clear structural markers.

Semantic-Based Chunking

Semantic-based chunking is the most sophisticated approach. You divide text into sentences, then use natural language processing to determine how related consecutive sentences are. You build chunks from groups of related sentences.

This method is computationally expensive but produces the most relevant chunks. It requires understanding the meaning of individual sentences and is more complex to implement than the other strategies.

Sentence-Based Chunking

A practical middle ground is chunking by sentences. You split the text into individual sentences using regular expressions, then group them into chunks with optional overlap:

```
def chunk_by_sentence(text, max_sentences_per_chunk=5,
overlap_sentences=1):
    sentences = re.split(r"(?<=[.!?])\s+", text)

    chunks = []
    start_idx = 0

    while start_idx < len(sentences):
        end_idx = min(start_idx + max_sentences_per_chunk,
len(sentences))
        current_chunk = sentences[start_idx:end_idx]
        chunks.append(" ".join(current_chunk))

        start_idx += max_sentences_per_chunk -
overlap_sentences

    if start_idx < 0:
        start_idx = 0
```

```
return chunks
```

Choosing Your Strategy

Your choice depends entirely on your use case and document guarantees:

- Structure-based: Best results when you control document formatting (like internal company reports)
- Sentence-based: Good middle ground for most text documents
- Size-based: Most reliable fallback that works with any content type, including code

Size-based chunking with overlap is often the go-to choice in production because it's simple, reliable, and works with any document type. While it may not give perfect results, it consistently produces reasonable chunks that won't break your pipeline.

Remember: there's no single "best" chunking strategy. The right approach depends on your specific documents, use cases, and the trade-offs you're willing to make between implementation complexity and chunk quality.

Text embeddings

Downloads

002_embeddings.ipynb

(opens in new tab)

VoyageAI API Key Directions.pdf

After breaking a document into chunks, the next step in a RAG pipeline is finding which chunks are most relevant to a user's question. This is essentially a search problem - you need to look through all your text chunks and identify the ones that relate to what the user is asking about.

Finding Relevant Chunks

- This is a search problem!
- How do we find chunks most related to the user's question?

ANTHROPIC

User's Question	How many bugs did engineers fix this year?
-----------------	--

Chunks of Text

This year our company engaged in many areas of research.

Section 1: Medical Research

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

Section 2: Software Engineering

This division dedicated significant effort to studying various infection vectors in our distributed systems

Semantic Search

The most common approach for finding relevant chunks is semantic search. Unlike keyword-based search that looks for exact word matches, semantic search uses text embeddings to understand the meaning and context of both the user's question and each text chunk.

Semantic Search

Utilize text embeddings to better understand the user's question *and* what each chunk of text is really talking about.

ANTHROPIC

User's Question	How many bugs did engineers fix this year?
-----------------	--

Chunks of Text

This year our company engaged in many areas of research.

```
## Section 1: Medical Research

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.
```

```
## Section 2: Software Engineering

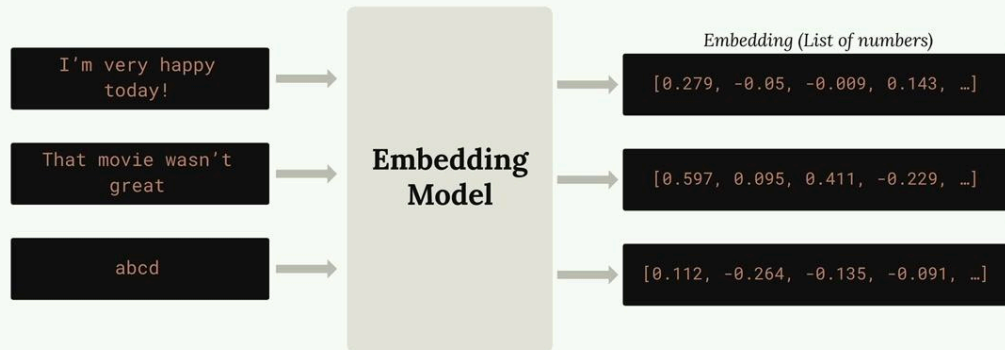
This division dedicated significant effort to studying various infection vectors in our distributed systems
```

Text Embeddings

A text embedding is a numerical representation of the meaning contained in some text. Think of it as converting words and sentences into a format that computers can work with mathematically.

Text Embeddings

A numerical representation of the meaning contained in some text.



ANTHROPIC

Here's how the process works:

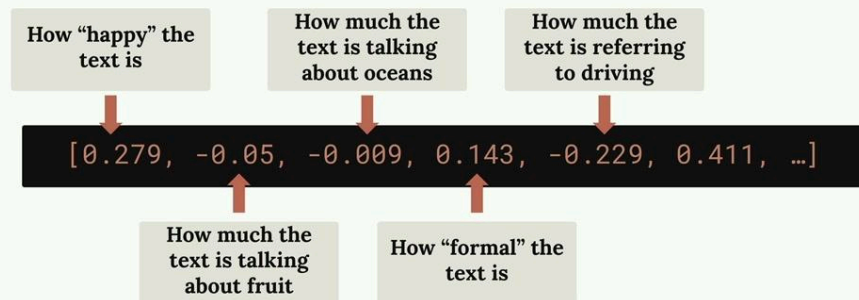
- You feed text into an embedding model
- The model outputs a long list of numbers (the embedding)
- Each number ranges from -1 to +1
- These numbers represent different qualities or features of the input text

Understanding the Numbers

Each number in an embedding is essentially a "score" for some quality of the input text. However, here's the important caveat: we don't know precisely what each number represents.

Text Embeddings

Each number is a “score” of some quality of the input text.
We don’t know precisely what each number represents.



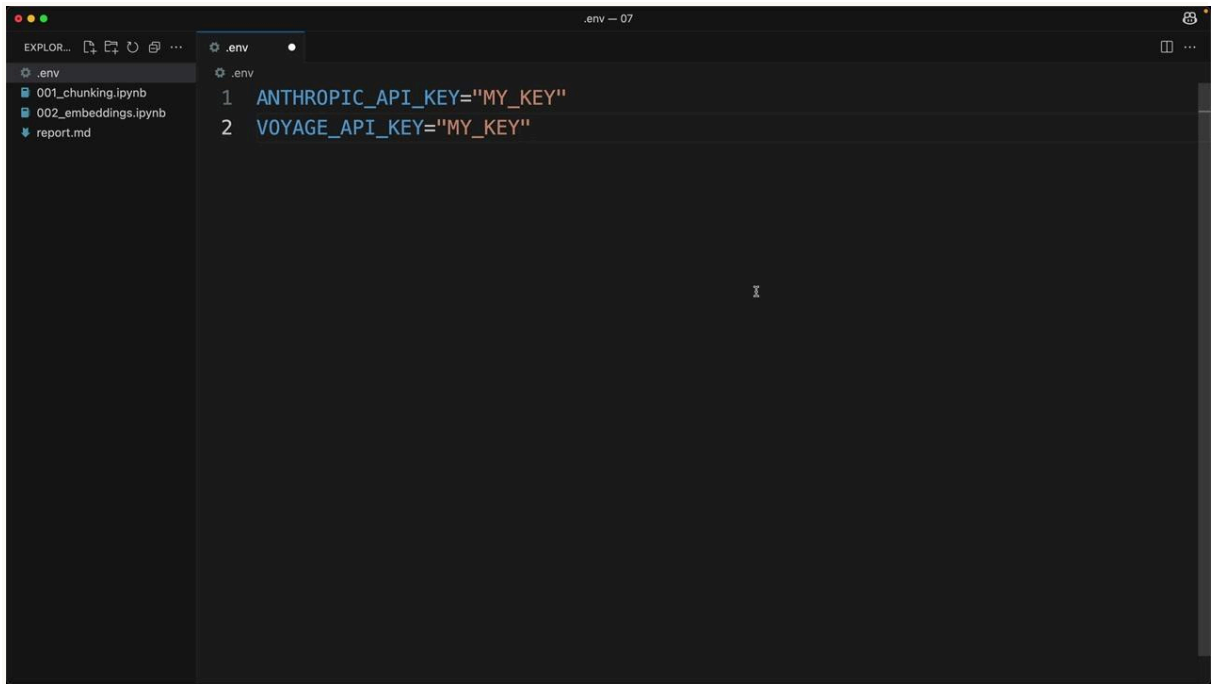
ANTHROPIC

While it's helpful to imagine that one number might represent "how happy the text is" or "how much the text talks about oceans," these are just conceptual examples. The actual meaning of each dimension is learned by the model during training and isn't directly interpretable by humans.

VoyageAI for Embeddings

Since Anthropic doesn't currently provide embedding generation, the recommended provider is VoyageAI. You'll need to:

- Sign up for a separate VoyageAI account
- Get an API key (free to get started)
- Add the key to your environment variables



In your **.env** file, add:

```
VOYAGE_API_KEY="your_key_here"
```

Implementation

First, install the VoyageAI library:

```
%pip install voyageai
```

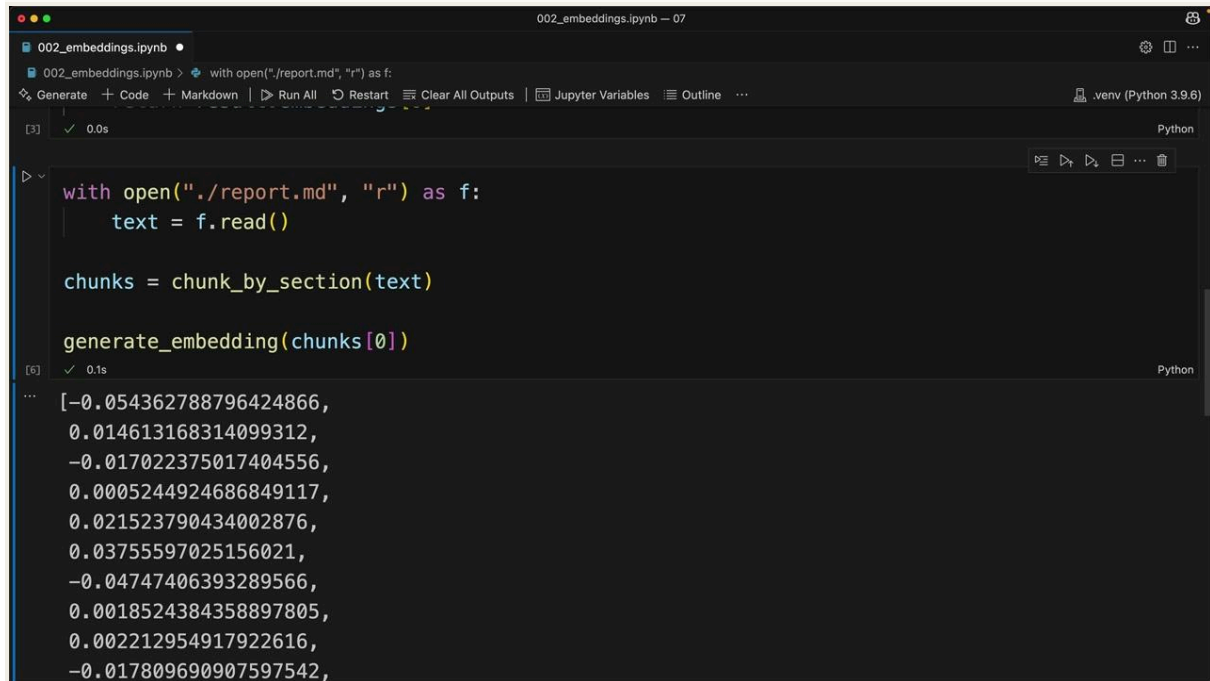
Then set up the client and create a function to generate embeddings:

```
from dotenv import load_dotenv  
import voyageai
```

```
load_dotenv()  
client = voyageai.Client()
```

```
def generate_embedding(text, model="voyage-3-large",
input_type="query"):
    result = client.embed([text], model=model,
input_type=input_type)

    return result.embeddings[0]
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The notebook is titled "002_embeddings.ipynb - 07". The code cell [3] contains the following Python code:

```
with open("./report.md", "r") as f:
    text = f.read()

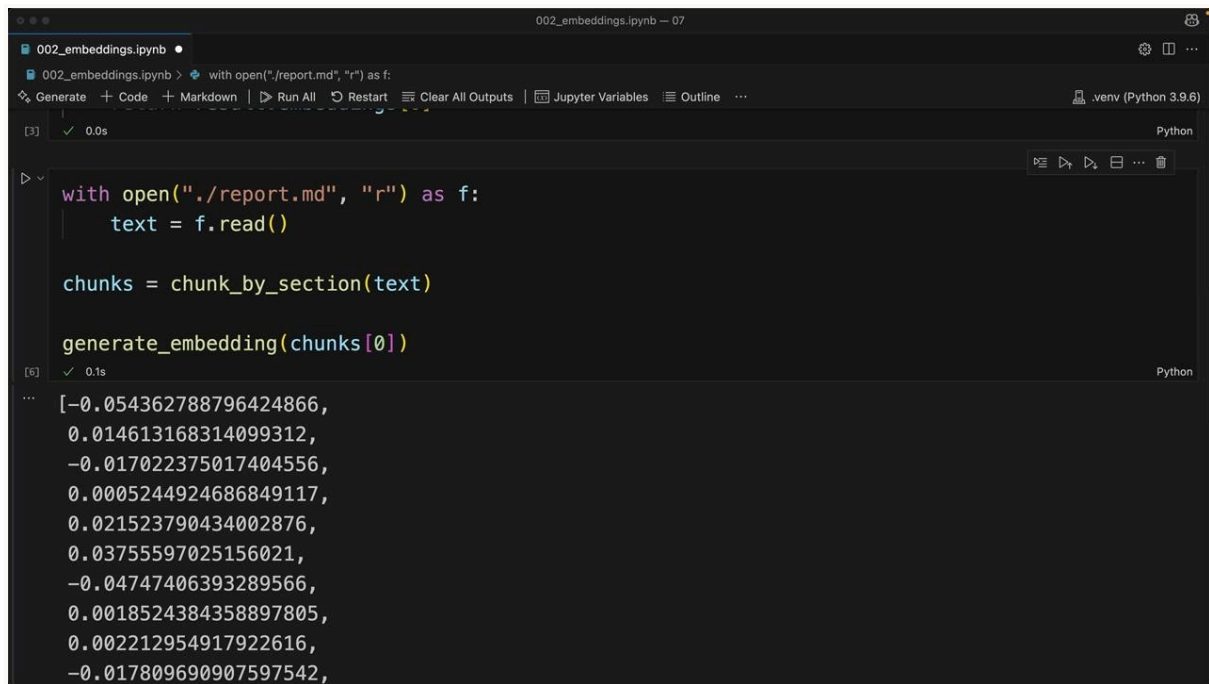
chunks = chunk_by_section(text)

generate_embedding(chunks[0])
```

The code cell [6] shows the output of the function, which is a list of floating-point numbers representing the embedding:

```
[[-0.054362788796424866,
 0.014613168314099312,
 -0.017022375017404556,
 0.0005244924686849117,
 0.021523790434002876,
 0.03755597025156021,
 -0.04747406393289566,
 0.0018524384358897805,
 0.002212954917922616,
 -0.017809690907597542,
```

When you run this function on a text chunk, you'll get back a list of floating-point numbers representing the embedding. The process is quick and straightforward - the real challenge is understanding how to use these embeddings effectively in your RAG pipeline for finding the most relevant content.



The screenshot shows a Jupyter Notebook interface. The top bar indicates the file is '002_embeddings.ipynb' and the environment is '.venv (Python 3.9.6)'. The code cell contains the following Python code:

```
with open("./report.md", "r") as f:
    text = f.read()

chunks = chunk_by_section(text)

generate_embedding(chunks[0])
```

The output cell shows a list of 11 floating-point numbers, representing an embedding vector:

```
[ -0.054362788796424866,
  0.014613168314099312,
 -0.017022375017404556,
  0.0005244924686849117,
  0.021523790434002876,
  0.03755597025156021,
 -0.04747406393289566,
  0.0018524384358897805,
  0.002212954917922616,
 -0.017809690907597542,
```

The next step is learning how to compare embeddings to determine which chunks are most similar to a user's question, which forms the core of the semantic search process.

Downloads

[002_embeddings.ipynb](#)

(opens in new tab)

[VoyageAI API Key Directions.pdf](#)

(opens in new tab)

The full RAG flow

Now that we've covered the basics of RAG, text chunking, and embeddings, let's walk through the complete RAG pipeline step by step. This example will show you exactly how all these pieces work together to retrieve relevant information and generate responses.

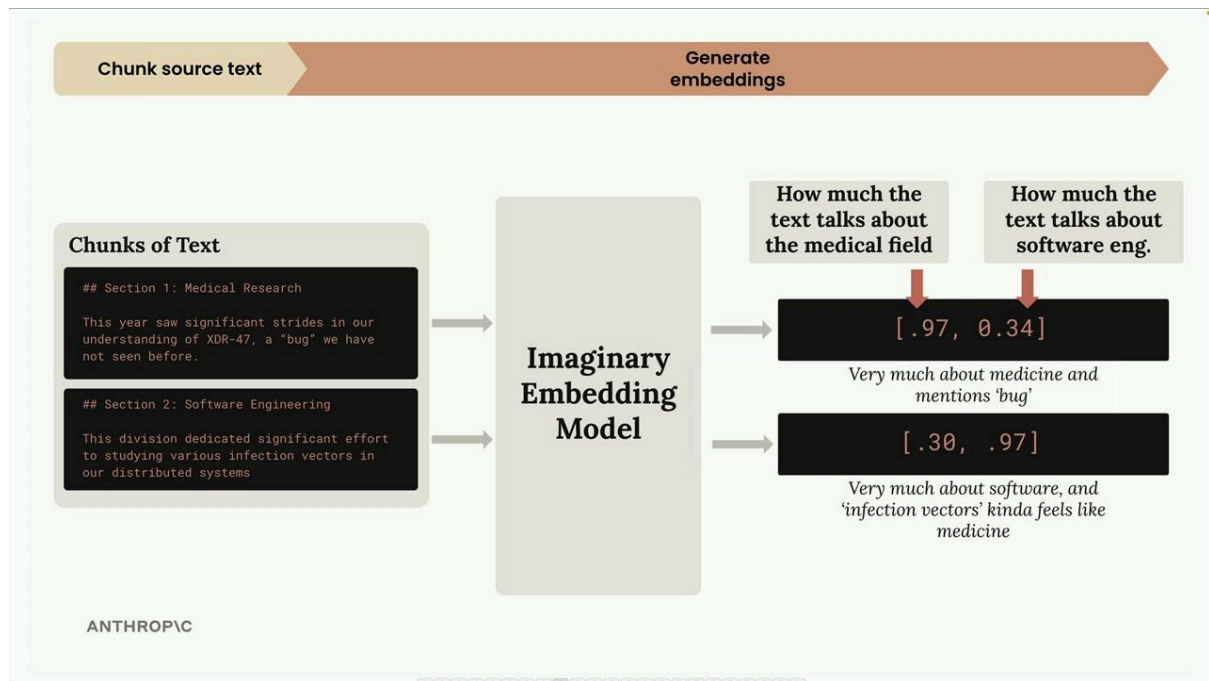
Step 1: Chunk Your Source Text

First, we take our source document and break it into manageable chunks. For this example, we'll use two simple text sections:

- Section 1: Medical Research - "This year saw significant strides in our understanding of XDR-47, a 'bug' we have not seen before."
- Section 2: Software Engineering - "This division dedicated significant effort to studying various infection vectors in our distributed systems"

Step 2: Generate Embeddings

Next, we convert each text chunk into numerical embeddings using an embedding model. To make this easier to understand, let's imagine we have a perfect embedding model that always returns exactly two numbers, and we know what each number represents.



In our imaginary model:

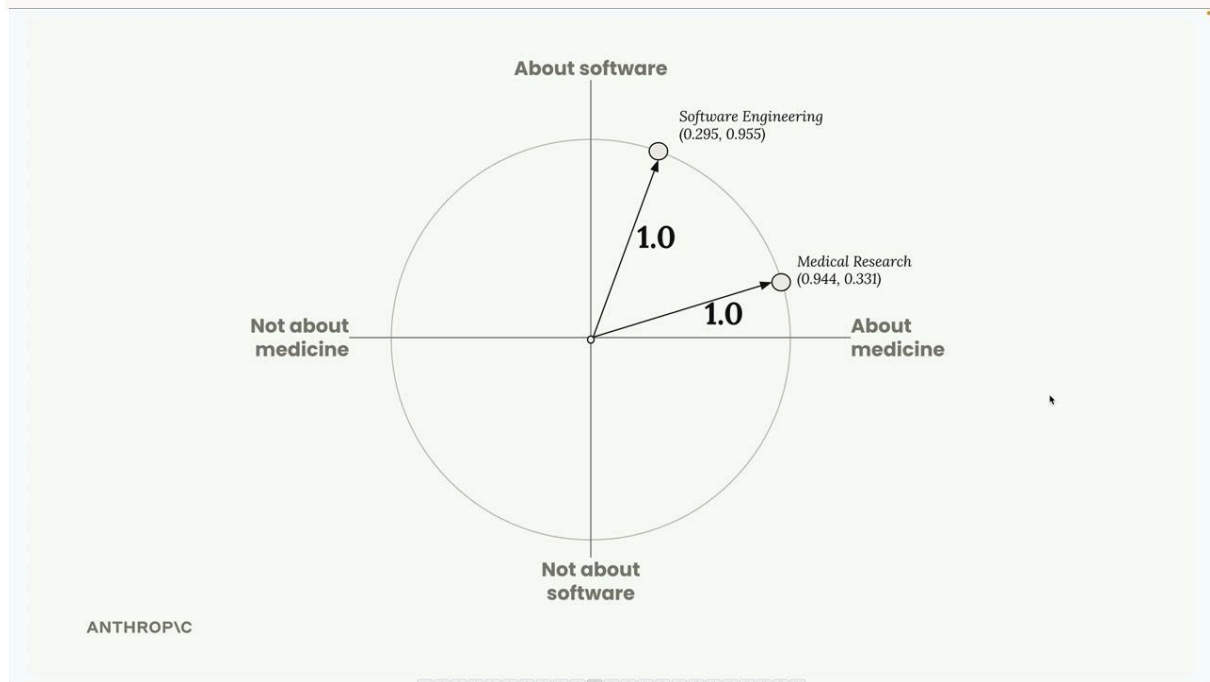
- The first number represents how much the text talks about the medical field
- The second number represents how much the text talks about software engineering

For the medical research section, we might get $[0.97, 0.34]$ - very medical-focused but with some software elements due to the word "bug". For the software engineering section, we get $[0.30, 0.97]$ - heavily software-focused but with medical undertones from "infection vectors".

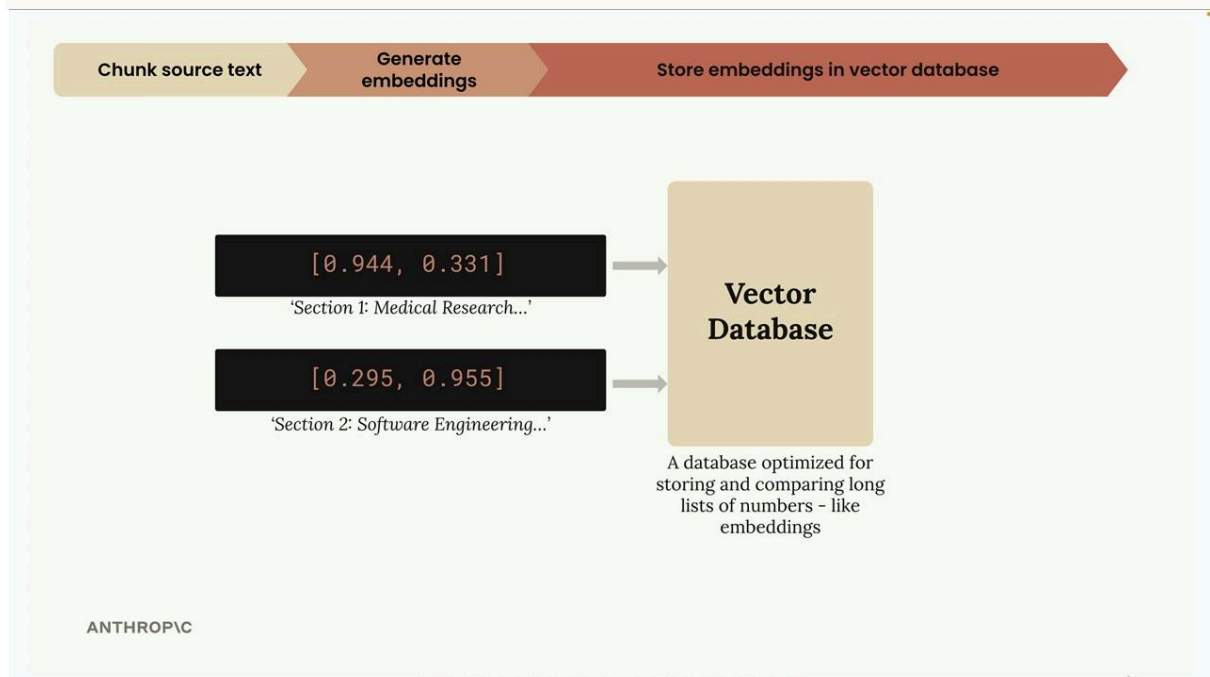
Normalization

The embedding API typically performs a normalization step that scales each vector to have a magnitude of 1.0. You don't need to worry about the

math here - it's handled automatically. This gives us normalized vectors like $[0.944, 0.331]$ and $[0.295, 0.955]$.

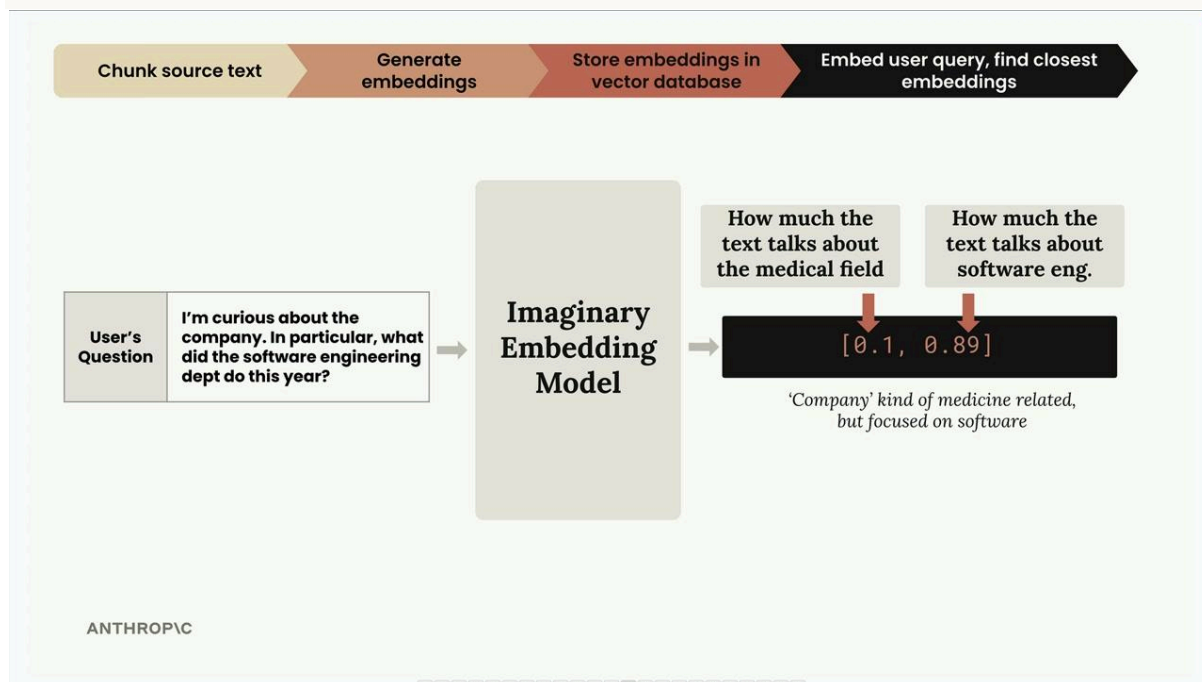


We can visualize these embeddings on a unit circle, where each point represents one of our text chunks.



Step 3: Store in Vector Database

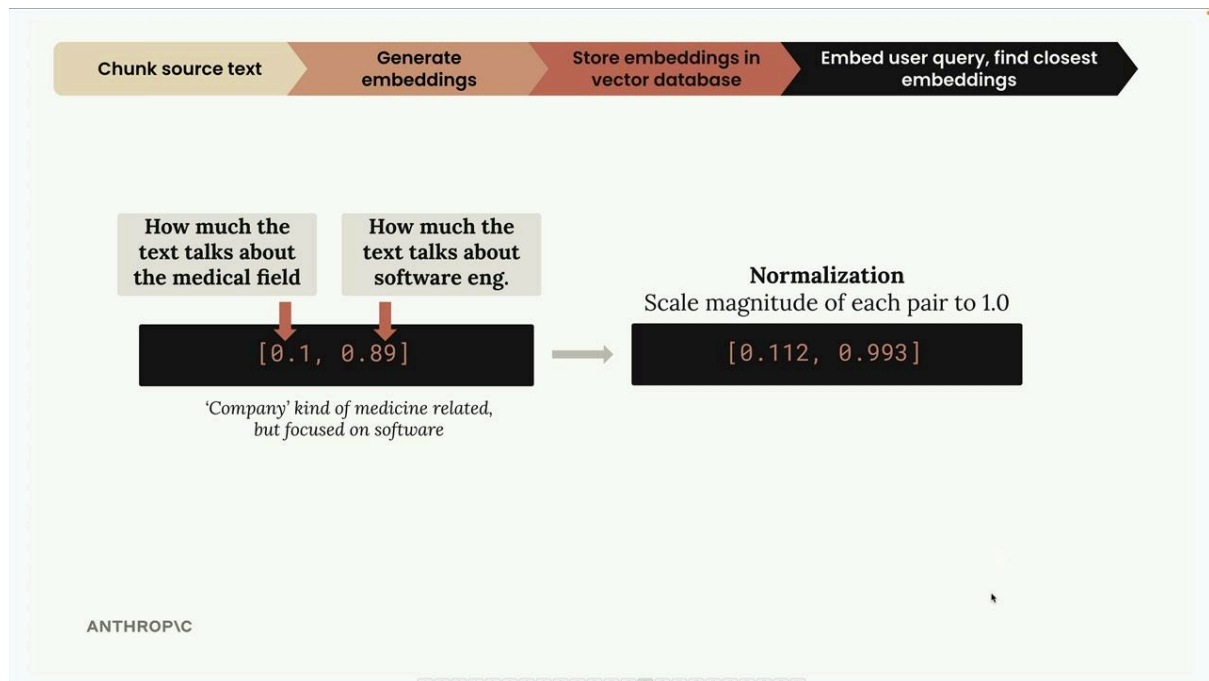
We store these embeddings in a vector database - a specialized database optimized for storing, comparing, and searching through long lists of numbers like our embeddings.



At this point, we pause. All the work so far has been preprocessing that happens ahead of time. Now we wait for a user to submit a query.

Step 4: Process User Query

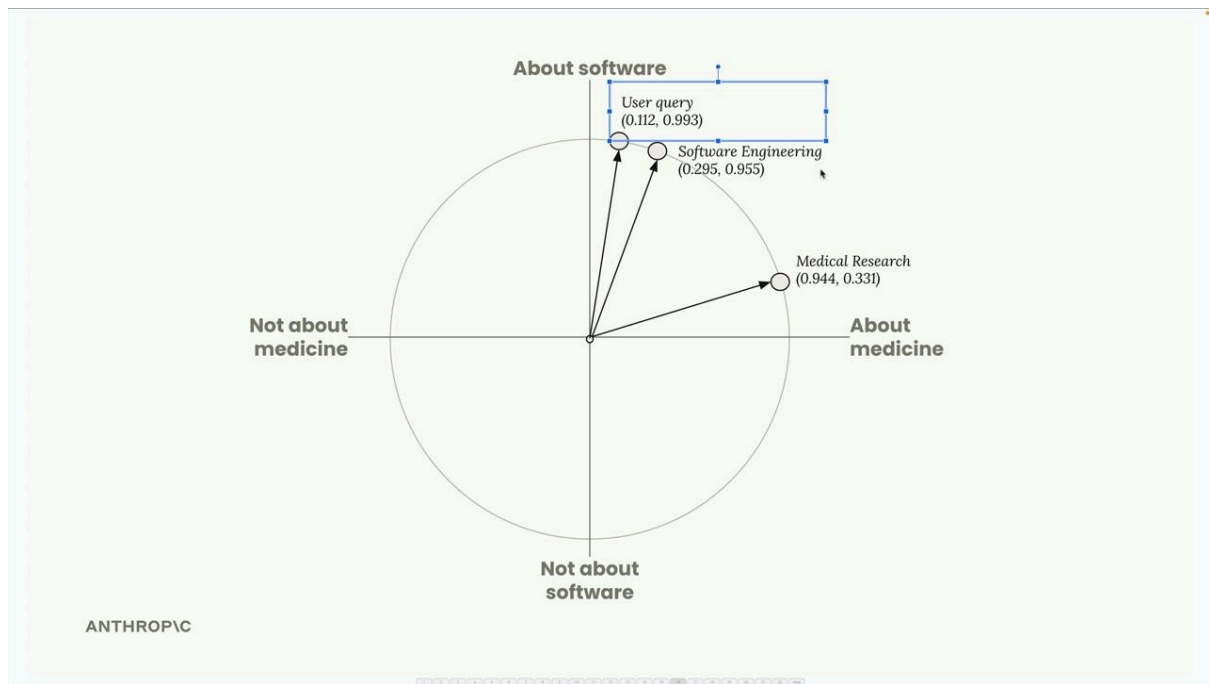
When a user asks a question like "I'm curious about the company. In particular, what did the software engineering dept do this year?", we run their query through the same embedding model.



This query gets embedded as something like $[0.1, 0.89]$ - low medical score, high software engineering score. After normalization, we get $[0.112, 0.993]$.

Step 5: Find Similar Embeddings

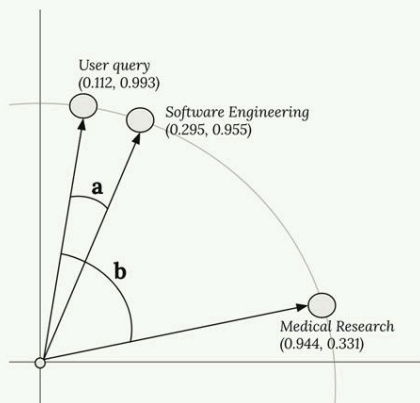
We send the user's query embedding to our vector database and ask it to find the most similar stored embeddings.



The database returns the software engineering section because it's the closest match to what the user asked about.

How Similarity Works: Cosine Similarity

The vector database uses cosine similarity to determine which embeddings are most similar. This measures the cosine of the angle between two vectors.



Cosine Similarity

- Cosine of the angle between the two vectors
- If they're in the exact same direction we'd get 1.0
- If perpendicular, 0.0
- If completely opposite, -1.0

$$\cos(a) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{(0.112, 0.993) \cdot (0.295, 0.955)}{\|(0.112, 0.993)\| \cdot \|(0.295, 0.955)\|} = 0.983$$

$$\cos(b) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{(0.112, 0.993) \cdot (0.944, 0.331)}{\|(0.112, 0.993)\| \cdot \|(0.944, 0.331)\|} = 0.398$$

Key points about cosine similarity:

- Results range from -1 to 1
- Values close to 1 mean high similarity
- Values close to -1 mean very different
- 0 means perpendicular (no relationship)

In our example, the cosine similarity between the user query and the software engineering chunk is 0.983 - very high similarity. The similarity with the medical research chunk is only 0.398 - much lower.

Cosine Distance

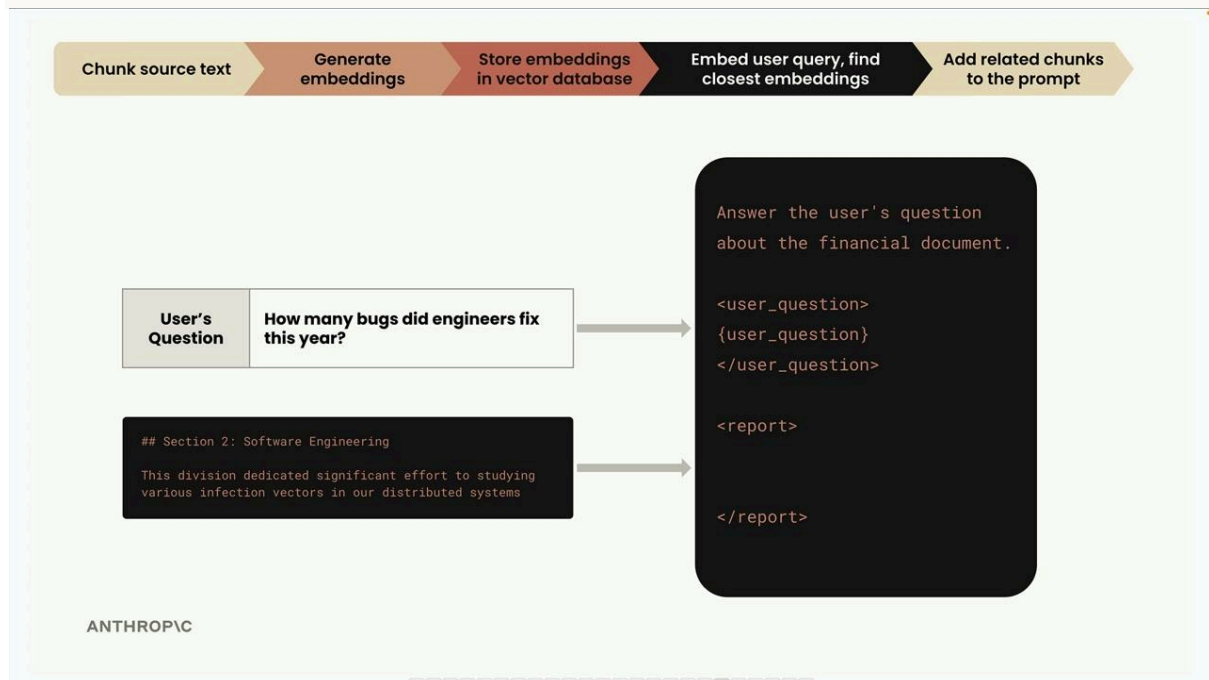
You'll often see "cosine distance" in vector database documentation. This is simply calculated as **(1 - cosine similarity)**. With cosine distance:

- Values close to 0 mean high similarity
- Larger values mean less similarity

This adjustment makes the numbers easier to interpret in many contexts.

Step 6: Create the Final Prompt

Finally, we take the user's question and the most relevant text chunk we found, combine them into a prompt, and send it to Claude for a response.



The prompt might look like:

```
Answer the user's question about the financial document.
```

```
<user_question>  
How many bugs did engineers fix this year?  
</user_question>
```

```
<report>  
## Section 2: Software Engineering
```

This division dedicated significant effort to studying various infection vectors in our distributed systems

</report>

And that's the complete RAG pipeline! The system successfully retrieved the most relevant information based on semantic similarity and provided it as context for generating an accurate response.

Implementing the RAG flow

Downloads

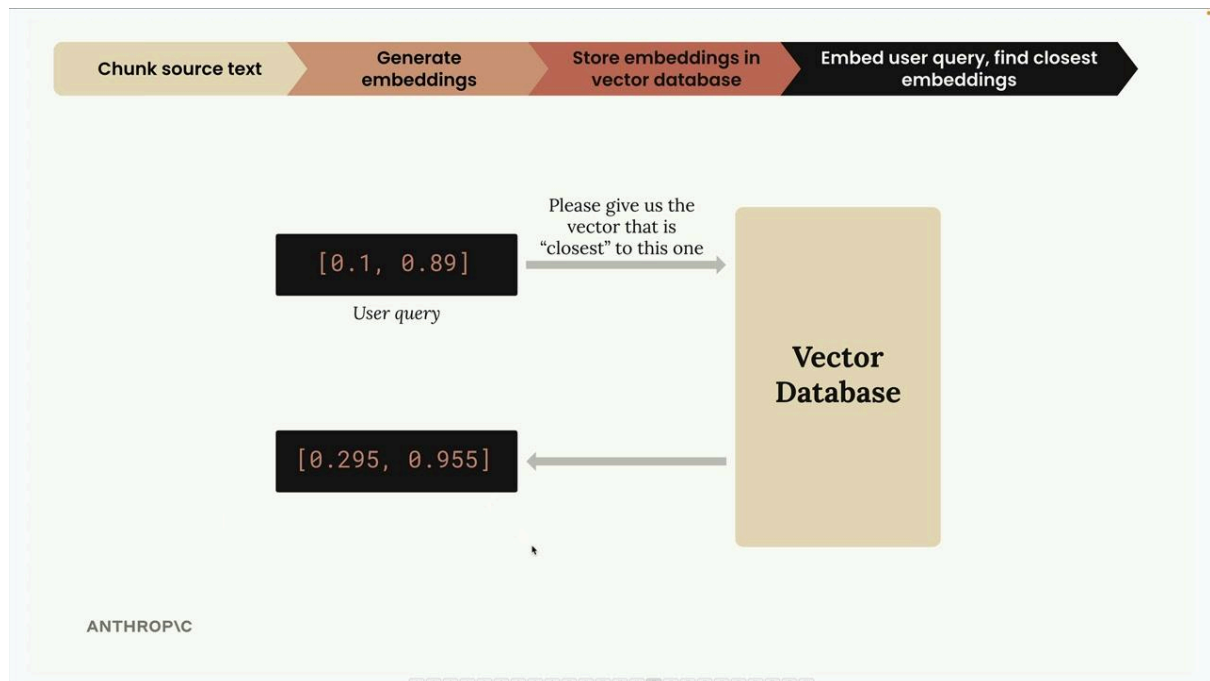
[003_vectordb.ipynb](#)

Now that we understand the RAG flow conceptually, let's implement it step by step. We'll walk through a complete example that demonstrates how to chunk text, generate embeddings, store them in a vector database, and perform similarity searches.

The Five-Step RAG Implementation

Our implementation follows the same five steps we discussed previously:

1. Chunk the text by section
2. Generate embeddings for each chunk
3. Create a vector store and add each embedding to it
4. Generate an embedding for the user's question
5. Search the store to find the most relevant chunks



This diagram shows how we transform user queries into embeddings and search our vector database to find the most relevant content.

Step 1: Chunking the Text

First, we load our document and split it into manageable sections:

```
with open("./report.md", "r") as f:
    text = f.read()

chunks = chunk_by_section(text)

chunks[2]  # Test to see the table of contents
```

We use the same **chunk_by_section** function from earlier to split our document into logical sections.

Step 2: Generate Embeddings

Next, we create embeddings for all our chunks at once:

```
embeddings = generate_embedding(chunks)
```

The embedding function has been updated to handle both single strings and lists of strings, making it more efficient for batch processing.

Step 3: Store in Vector Database

Now we create our vector store and populate it with embeddings and their associated text:

```
store = VectorIndex()
```

```
for embedding, chunk in zip(embeddings, chunks):
```

```
    store.add_vector(embedding, {"content": chunk})
```

Notice that we store both the embedding and the original text content. This is crucial because when we search later, we need to return the actual text, not just the numerical embedding values.

Why Store the Original Text?

When we query our vector database, getting back just the embedding numbers isn't useful. We need the actual text that was used to generate those embeddings. That's why we include the original chunk text (or at least a reference to it) alongside each embedding in our database.

Step 4: Process User Queries

When a user asks a question, we generate an embedding for their query:

```
user_embedding = generate_embedding("What did the software engineering dept do last year?")
```

Step 5: Find Relevant Content

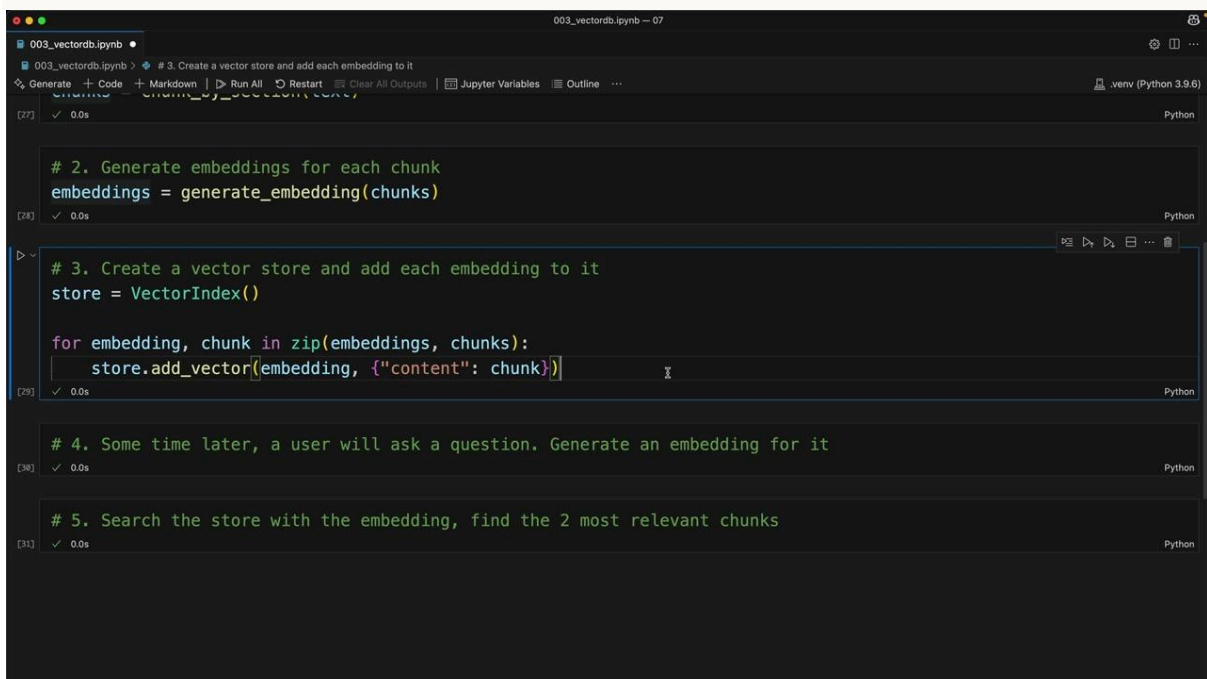
Finally, we search our vector store to find the most similar chunks:

```
results = store.search(user_embedding, 2)

for doc, distance in results:

    print(distance, "\n", doc["content"][0:200], "\n")
```

This search returns the two most relevant chunks along with their similarity scores (cosine distances).



The screenshot shows a Jupyter Notebook titled '003_vectordb.ipynb' with the following code cells:

```
# 2. Generate embeddings for each chunk
embeddings = generate_embedding(chunks)

# 3. Create a vector store and add each embedding to it
store = VectorIndex()

for embedding, chunk in zip(embeddings, chunks):
    store.add_vector(embedding, {"content": chunk})

# 4. Some time later, a user will ask a question. Generate an embedding for it

# 5. Search the store with the embedding, find the 2 most relevant chunks
```

The notebook interface includes a toolbar with options like 'Generate', 'Code', 'Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Jupyter Variables', and 'Outline'. The code is written in Python and is being executed in a Jupyter environment.

The search results show us which sections of our document are most relevant to the user's question, along with similarity scores.

Understanding the Results

When we run our example query about the software engineering department, we get back:

- Section 2: Software Engineering with a distance of 0.71 (closest match)
- Methodology section with a distance of 0.72 (second closest)

Lower distance values indicate higher similarity, so Section 2 is the most relevant to our query.

What's Next?

This implementation works well for basic cases, but there are scenarios where it doesn't perform as expected. In the next sections, we'll explore improvements to make our RAG system more robust and accurate.

The key takeaway is that RAG is fundamentally about converting text to numbers (embeddings), storing those numbers efficiently, and then using mathematical similarity to find relevant content when users ask questions.

BM25 lexical search

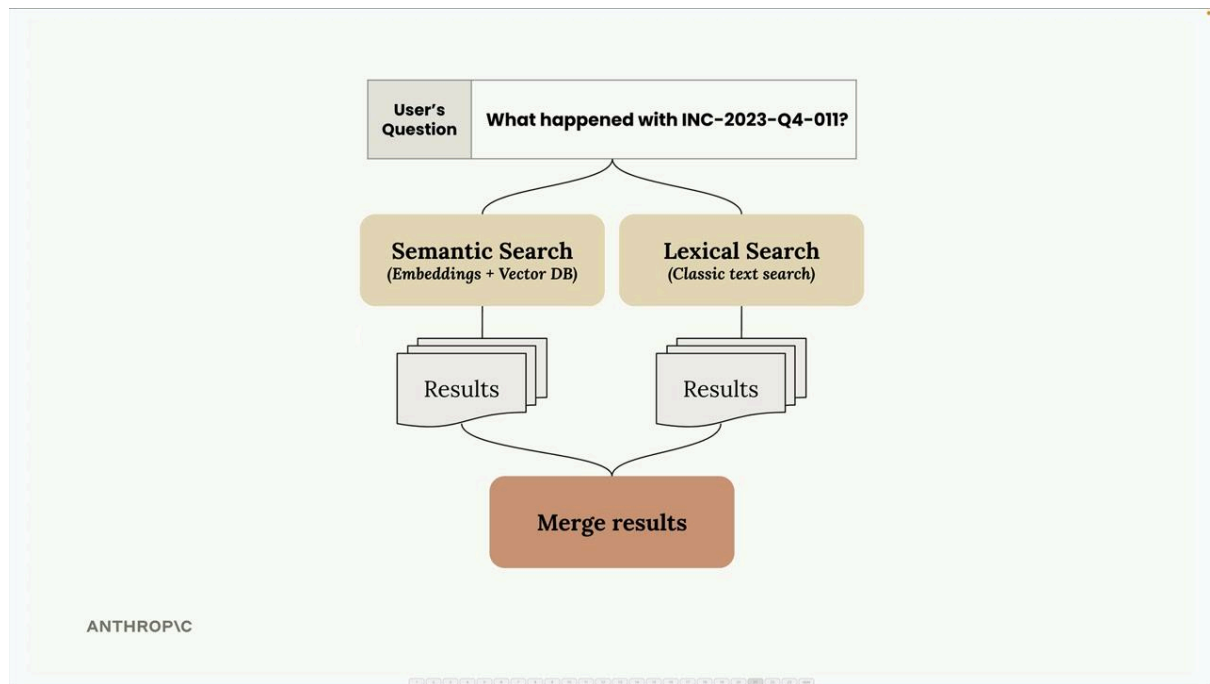
Downloads

004_bm25.ipynb

When building RAG pipelines, you'll quickly discover that semantic search alone doesn't always return the best results. Sometimes you need exact term matches that semantic search might miss. The solution is to combine semantic search with lexical search using a technique called BM25.

The Problem with Semantic Search Alone

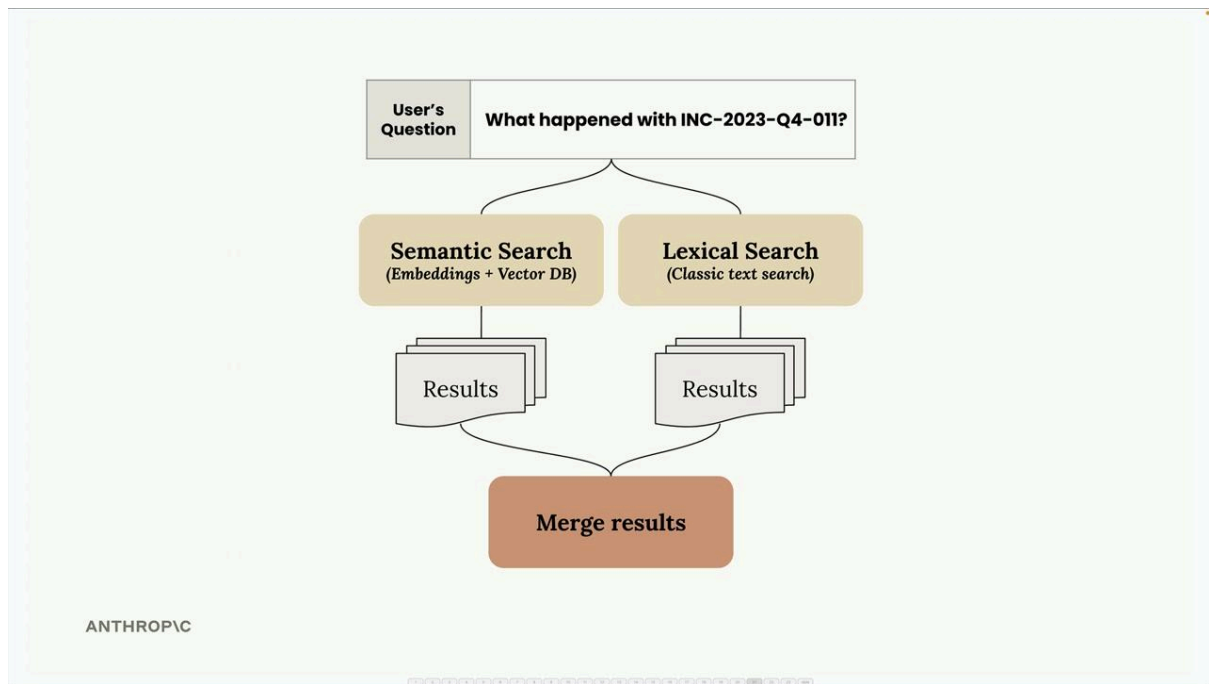
Let's say you're searching for a specific incident ID like "INC-2023-Q4-011" in a document. While semantic search excels at understanding context and meaning, it might return sections that are semantically related but don't actually contain the exact term you're looking for.



In the example above, semantic search returned the cybersecurity section (which does contain the incident ID) but also returned a financial analysis section that doesn't mention the incident at all. This happens because semantic search focuses on conceptual similarity rather than exact term matching.

Hybrid Search Strategy

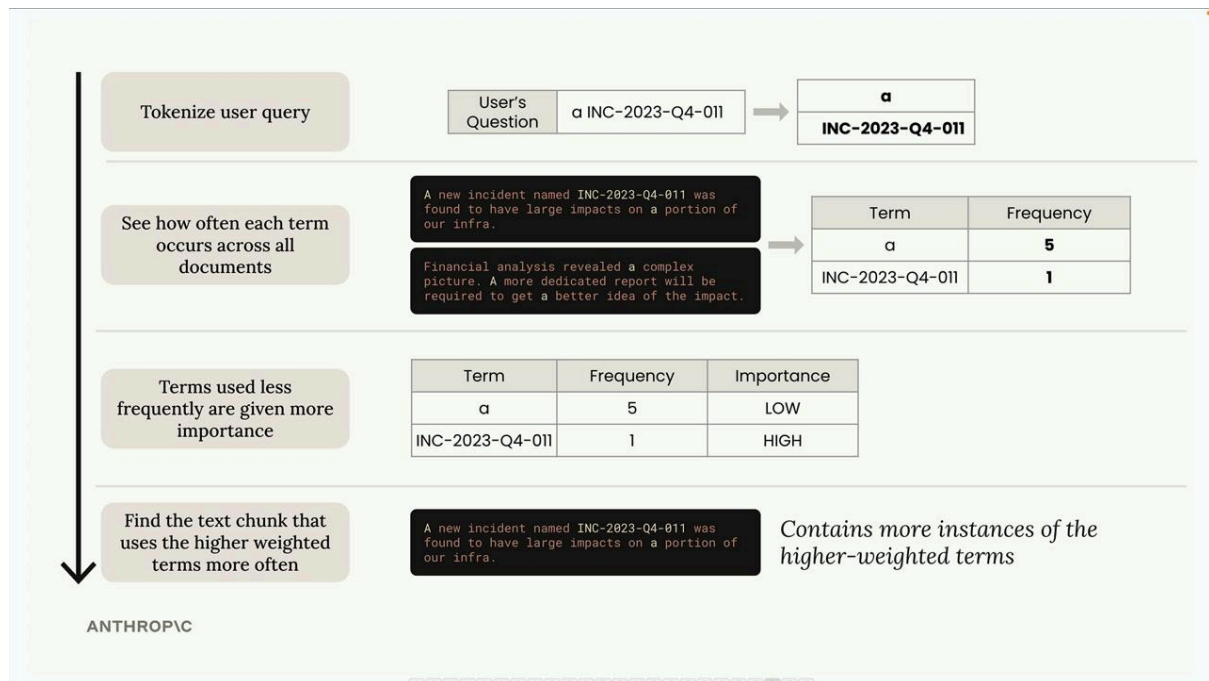
The solution is to run both semantic and lexical searches in parallel, then merge the results. This gives you the best of both worlds:



- Semantic search finds conceptually related content using embeddings
- Lexical search finds exact term matches using classic text search
- Merged results combine both approaches for better accuracy

How BM25 Works

BM25 (Best Match 25) is a popular algorithm for lexical search in RAG systems. Here's how it processes a search query:



Step 1: Tokenize the query

Break the user's question into individual terms. For example, "a INC-2023-Q4-011" becomes ["a", "INC-2023-Q4-011"].

Step 2: Count term frequency

See how often each term appears across all your documents. Common words like "a" might appear 5 times, while specific terms like "INC-2023-Q4-011" might appear only once.

Step 3: Weight terms by importance

Terms that appear less frequently get higher importance scores. The word "a" gets low importance because it's common, while "INC-2023-Q4-011" gets high importance because it's rare.

Step 4: Find best matches

Return documents that contain more instances of the higher-weighted terms.

Implementing BM25 Search

Here's how to set up a basic BM25 search system:

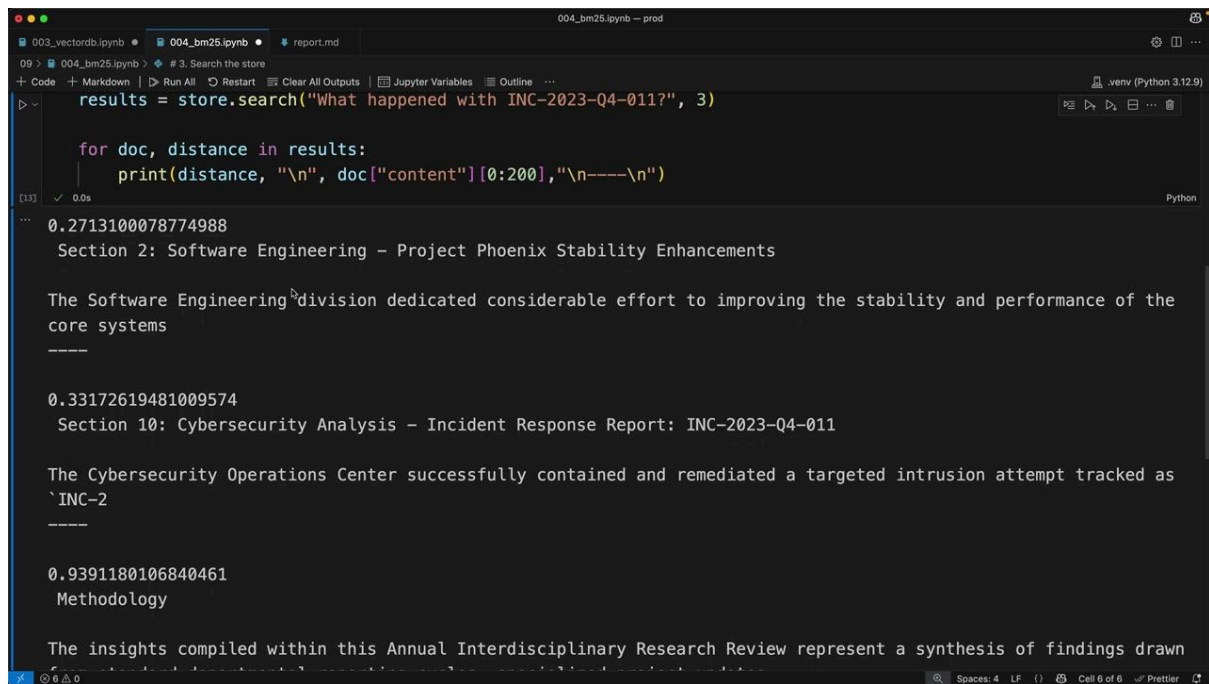
```
# 1. Chunk your text by sections
chunks = chunk_by_section(text)

# 2. Create a BM25 store and add documents
store = BM25Index()
for chunk in chunks:
    store.add_document({"content": chunk})

# 3. Search the store
results = store.search("What happened with INC-2023-Q4-011?",
3)

# Print results
for doc, distance in results:
    print(distance, "\n", doc["content"][:200], "\n---\n")
```

When you run this search, you'll get much better results than semantic search alone. The BM25 algorithm prioritizes sections that actually contain your specific search terms, especially rare terms like incident IDs.

A screenshot of a Jupyter Notebook interface. The top bar shows the file name '004_bm25.ipynb' and the environment 'venv (Python 3.12.9)'. The code cell contains a search query: `results = store.search("What happened with INC-2023-Q4-011?", 3)` followed by a loop that prints the distance and content of the top 3 results. The output shows three results, each with a distance score and a text snippet. The first result is 'Section 2: Software Engineering - Project Phoenix Stability Enhancements' with a distance of 0.2713100078774988. The second result is 'Section 10: Cybersecurity Analysis - Incident Response Report: INC-2023-Q4-011' with a distance of 0.33172619481009574. The third result is 'Methodology' with a distance of 0.9391180106840461. The text snippets are truncated with '-----' at the end.

```
results = store.search("What happened with INC-2023-Q4-011?", 3)

for doc, distance in results:
    print(distance, "\n", doc["content"][0:200], "\n-----\n")
```

0.2713100078774988
Section 2: Software Engineering - Project Phoenix Stability Enhancements

The Software Engineering division dedicated considerable effort to improving the stability and performance of the core systems

0.33172619481009574
Section 10: Cybersecurity Analysis - Incident Response Report: INC-2023-Q4-011

The Cybersecurity Operations Center successfully contained and remediated a targeted intrusion attempt tracked as `INC-2

0.9391180106840461
Methodology

The insights compiled within this Annual Interdisciplinary Research Review represent a synthesis of findings drawn

Notice how the results now properly prioritize the Software Engineering section and Cybersecurity section - both of which actually contain the incident ID you're searching for.

Why This Works Better

BM25 excels at finding exact matches because it:

- Gives higher weight to rare, specific terms
- Ignores common words that don't add search value
- Focuses on term frequency rather than semantic meaning
- Works especially well for technical terms, IDs, and specific phrases

The key insight is that both search methods have complementary strengths. Semantic search understands context and meaning, while lexical search ensures you don't miss exact term matches. By combining them, you

create a more robust search system that handles both conceptual queries and specific lookups effectively.

In the next step, you'll learn how to merge results from both search systems to create a unified hybrid search experience.

A Multi-Index RAG pipeline

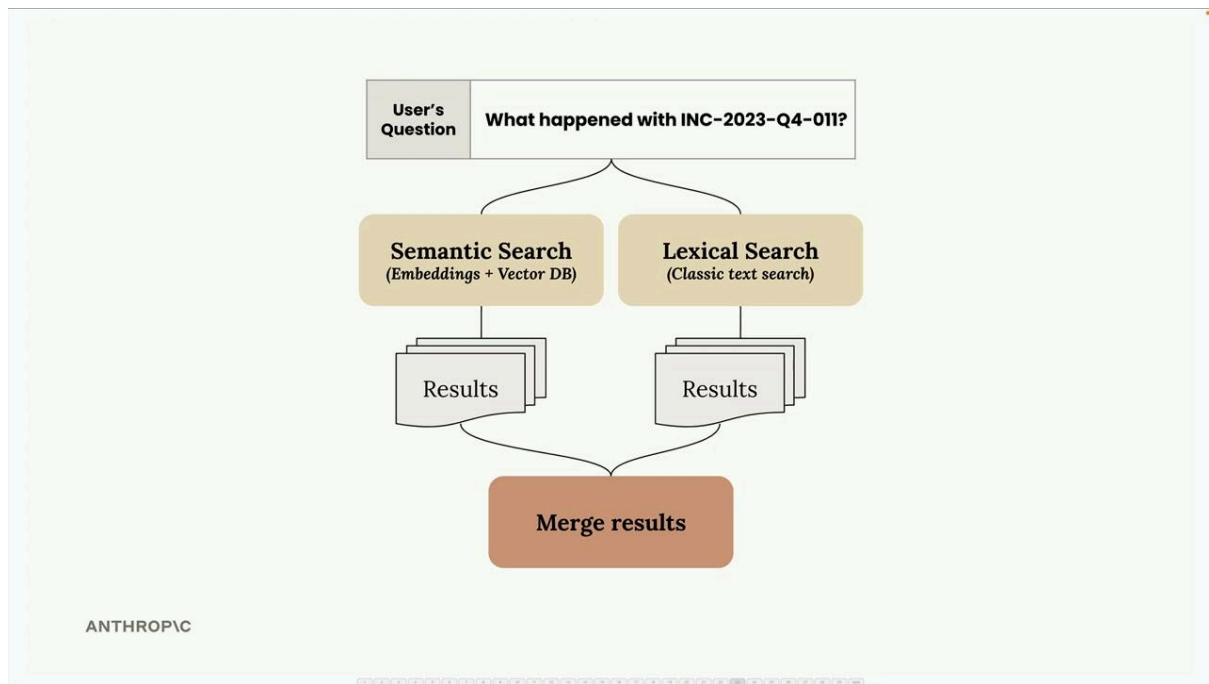
Downloads

005_hybrid.ipynb

We've built separate implementations for semantic search (using vector embeddings) and lexical search (using BM25). Now it's time to combine them into a unified search pipeline that leverages the strengths of both approaches.

The Multi-Index Architecture

Both our `VectorIndex` and `BM25Index` classes share nearly identical APIs - they both have `add_document()` and `search()` methods. This consistency makes it straightforward to wrap them together in a new class called `Retriever`.



The Retriever acts as a coordinator that forwards user queries to both indexes, collects their results, and merges them using a technique called reciprocal rank fusion.

Understanding Reciprocal Rank Fusion

Merging results from different search methods isn't as simple as just concatenating lists. Each method uses different scoring systems, so we need a way to normalize and combine their rankings fairly.

Reciprocal Rank Fusion

Combine outputs based upon the **reciprocal rank** of each text chunk

VectorIndex

Rank	Text Chunk	Score
1	Section 2: Software Engineering...	0.234
2	Section 7: Historical Research...	0.913
3	Section 6: Product Engineering	1.327

BM25Index

Rank	Text Chunk	Score
1	Section 6: Product Engineering	0.184
2	Section 2: Software Engineering...	0.414
3	Section 7: Historical Research...	0.781

ANTHROPIC

Here's how reciprocal rank fusion works with an example. Let's say we search for information about "INC-2023-Q4-011" and get these results:

- VectorIndex returns: Section 2 (rank 1), Section 7 (rank 2), Section 6 (rank 3)
- BM25Index returns: Section 6 (rank 1), Section 2 (rank 2), Section 7 (rank 3)

VectorIndex

Rank	Text Chunk
1	Section 2: Software Engineering...
2	Section 7: Historical Research...
3	Section 6: Product Engineering

BM25Index

Rank	Text Chunk
1	Section 6: Product Engineering
2	Section 2: Software Engineering...
3	Section 7: Historical Research...

Text Chunk	Rank From Vector Index	Rank from BM25 Index
Section 2: Software Engineering...	1	2
Section 7: Historical Research...	2	3
Section 6: Product Engineering...	3	1

ANTHROPIC

We combine these into a single table showing each text chunk's rank from both indexes, then apply the RRF formula:

$$\text{RRF_score}(d) = \sum (1 / (k + \text{rank}_i(d)))$$

Where k is a constant (often 60, but we'll use 1 for clearer results) and $\text{rank}_i(d)$ is the rank of document d in the i -th ranking.

$$\text{RRF_score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

k is a constant, 60 is often used, but going to use 1 here for clearer results

Text Chunk	Rank From Vector	Rank from BM25	Score
Section 2: Software Engineering...	1	2	$1.0 / (1 + 1) + 1.0 / (1 + 2) = \mathbf{0.833}$
Section 7: Historical Research...	2	3	$1.0 / (1 + 2) + 1.0 / (1 + 3) = \mathbf{0.583}$
Section 6: Product Engineering...	3	1	$1.0 / (1 + 3) + 1.0 / (1 + 1) = \mathbf{0.75}$

Higher score = more relevant text chunk

ANTHROPIC

For our example:

- Section 2: $1.0/(1+1) + 1.0/(1+2) = 0.833$
- Section 7: $1.0/(1+2) + 1.0/(1+3) = 0.583$
- Section 6: $1.0/(1+3) + 1.0/(1+1) = 0.75$

$$\text{RRF_score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

k is a constant, 60 is often used, but going to use 1 here for clearer results

Text Chunk	Rank From Vector	Rank from BM25	Score	Final Rank
Section 2: Software Engineering...	1	2	$1.0 / (1 + 1) + 1.0 / (1 + 2) = \mathbf{0.833}$	1
Section 6: Product Engineering...	3	1	$1.0 / (1 + 3) + 1.0 / (1 + 1) = \mathbf{0.75}$	2
Section 7: Historical Research...	2	3	$1.0 / (1 + 2) + 1.0 / (1 + 3) = \mathbf{0.583}$	3

Higher score = more relevant text chunk

ANTHROPIC

The final ranking becomes: Section 2 (0.833), Section 6 (0.75), Section 7 (0.583). This makes intuitive sense - Section 2 performed well in both indexes, so it rises to the top.

Implementation Details

The Retriever class wraps multiple search indexes and provides a unified interface:

```
class Retriever:
    def __init__(self, *indexes: SearchIndex):
        if len(indexes) == 0:
            raise ValueError("At least one index must be provided")
        self._indexes = list(indexes)

    def add_document(self, document: Dict[str, Any]):
        for index in self._indexes:
            index.add_document(document)
```

```
def search(self, query_text: str, k: int = 1, k_rrf: int = 60):  
    # Get results from all indexes  
    all_results = []  
    for idx, results in enumerate(all_results):  
        for rank, (doc, _) in enumerate(results):  
            # Track document ranks across indexes  
            # Apply RRF scoring formula  
  
    # Return merged and sorted results
```

The key insight is that by maintaining consistent APIs across different search implementations, we can easily combine them without tight coupling.

Testing the Hybrid Approach

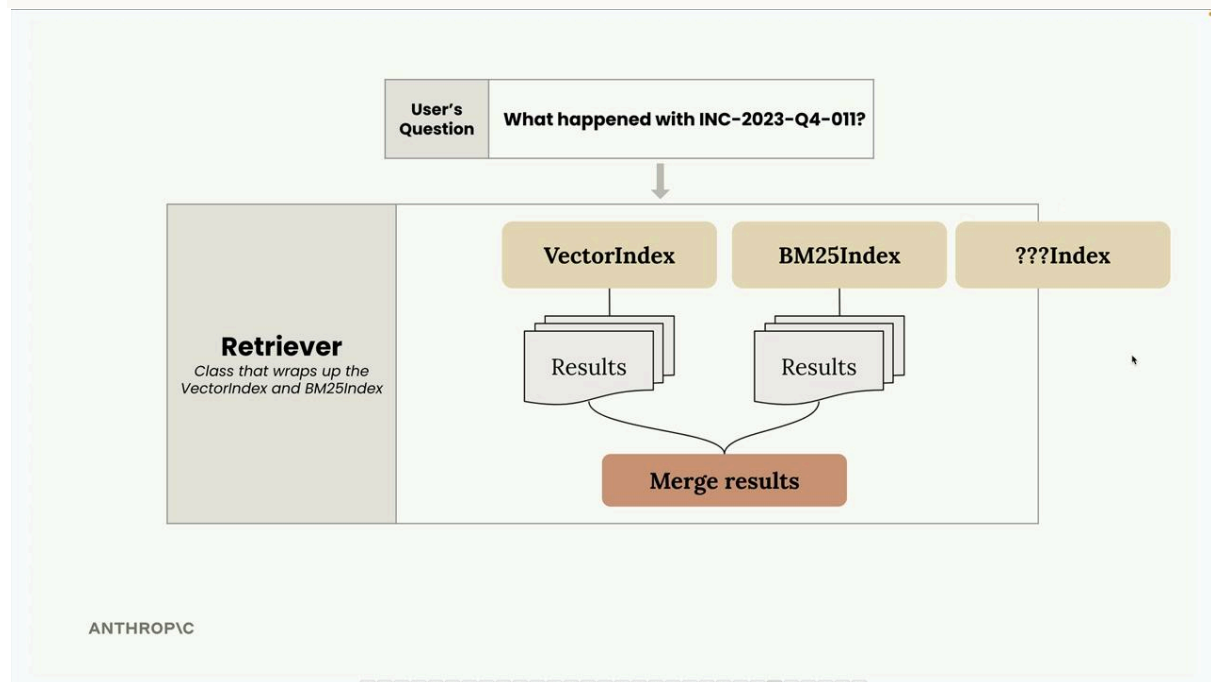
Remember our earlier problem where searching for "what happened with INC-2023-Q4-011?" returned unexpected results from the vector-only approach? The cybersecurity incident (Section 10) came first, but financial analysis (Section 3) came second instead of the more relevant software engineering section.

With our hybrid retriever, we now get much better results:

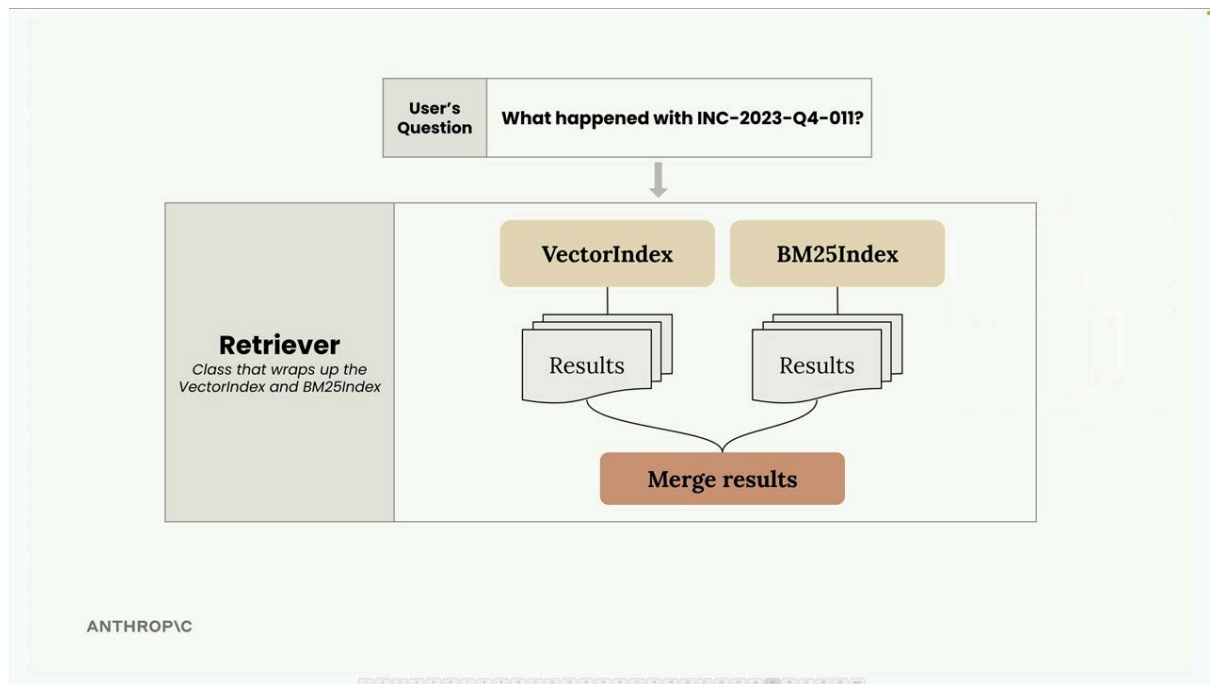
- Section 10: Cybersecurity Analysis - Incident Response Report (most relevant)
- Section 2: Software Engineering - Project Phoenix Stability Enhancements (second most relevant)
- Section 5: Legal Developments (third)

This demonstrates how combining semantic and lexical search can overcome the limitations of either approach used alone.

Extensibility



The beauty of this architecture is its extensibility. Since all indexes implement the same `SearchIndex` protocol with `add_document()` and `search()` methods, you can easily add new search methodologies:



Want to add a keyword-based index? A graph-based search? A specialized domain index? Just implement the same interface and the Retriever will automatically incorporate it into the fusion process.

This modular approach keeps each search implementation focused and testable while providing a clean way to combine their strengths in the final system.